

# BLE (2) - BLE 프로토콜 스택

Updated: April 14, 2020

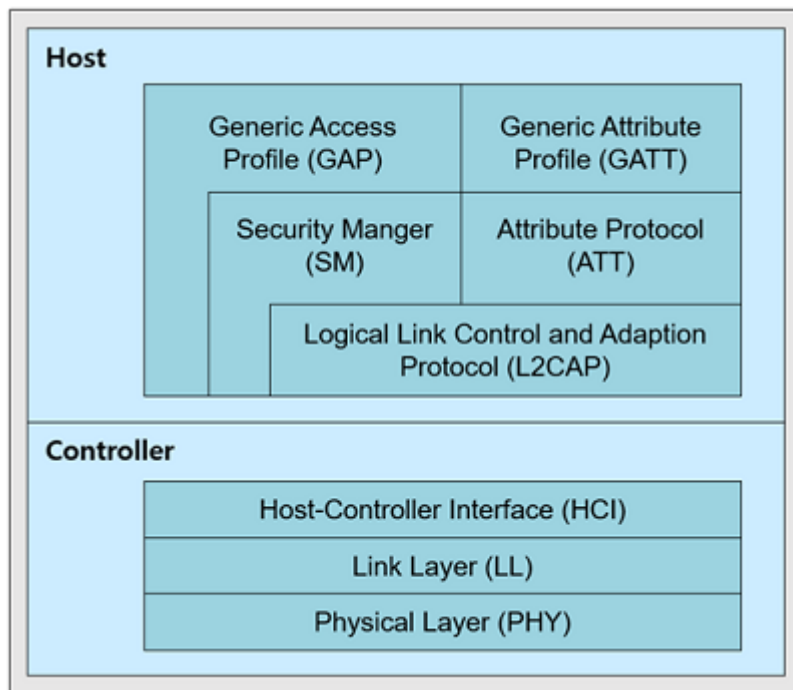
## On this page

- 2. BLE 프로토콜 스택
  - 2.1 Physical layer (PHY)
  - 2.2 Link layer (LL)
  - 2.3 HCI, L2CAP, SM
    - 2.3.1 HCI
    - 2.3.2 L2CAP
    - 2.3.3 SM
  - 2.4 Attribute Protocol (ATT)
  - 2.5 Generic Access Profile (GAP)

## 2. BLE 프로토콜 스택 [Permalink](#)

BLE 프로토콜 스택 (**BLE protocol stack, BLE stack**)은 BLE 디바이스의 구조를 보여주며, 이는 블루투스 표준 (**Bluetooth Core Specification**)에서 정의하고 있는 LE (**Low Energy**) 모드로 동작하는 디바이스가 갖춰야하는 기능들을 포함하고 있다.

*The BLE protocol stack implements all the mandatory and optional features of LE mode compliant to Core Specification*



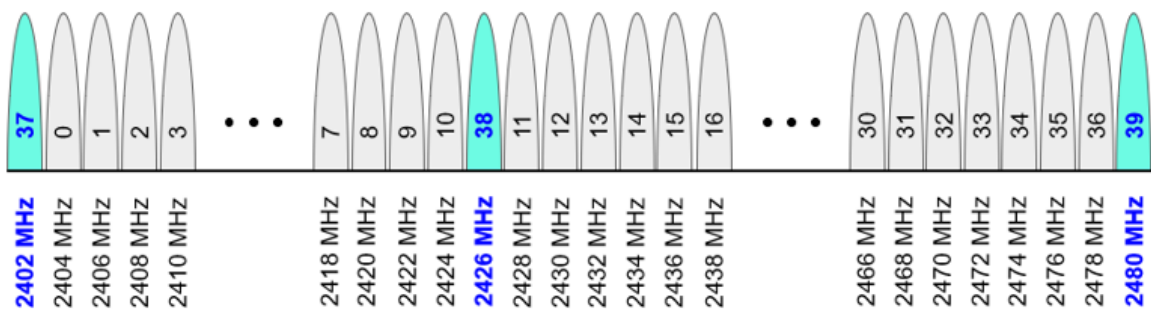
BLE 스택은 크게 **Host** 와 **Controller** 로 구성되어 있고, 각 영역은 다양한 종류의 레이어 (**layer**) 혹은 프로파일 (**profile**)로 이루어져있다. BLE 디바이스는 각 레이어에서 특정 상태(**state**) 혹은 역할(**role**)을 가지고 동작하게 되며, BLE 시스템을 개발하기전 BLE 스택의 각 요소가 어떤 역할을 하는지를 이해하는 것은 필수이다.

**Notice:** 다음의 서술하는 내용에서는 혼돈을 방지하기 위해 영어로 표기된 고유 명사의 대부분을 원문 영단어 그대로 사용합니다. 또한, 필자가 이해하고 있는 방식으로 바탕으로 서술하고 있으므로 용어나 표현에 있어서 잘못 서술하는 내용이 있을 수 있습니다.

## 2.1 Physical layer (PHY)Permalink

**PHY** 는 RF 시스템 및 무선 전파(**radio**) 신호를 송수신하기 위한 **하드웨어**로 구성된다. BLE 디바이스는 2.4 GHz 의 **ISM** 주파수 대역에서 동작하며, **GFSK** 과 **FHSS** 방식으로 데이터를 송수신한다.

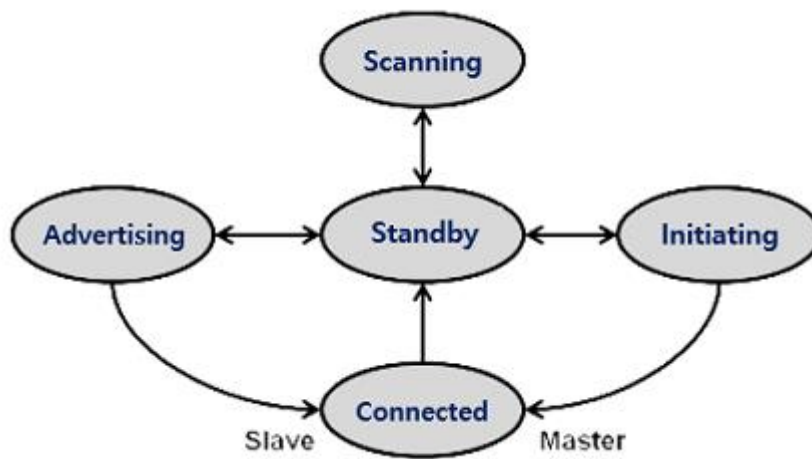
**PHY** 에서 BLE 시스템은 2.4 GHz 주파수 영역의 일부를 40 개의 구간으로 나누어 신호를 주고 받으며, 각각의 영역을 **채널**이라고 부른다. 각 채널은 중심 주파수를 기준으로 2 MHz 의 대역폭을 가지며, 할당된 40 개의 채널 중 37-39 번 채널은 **advertising** 채널로 사용되고, 나머지 37 개의 채널은 **data** 채널로 사용된다 (각 채널에 대해서는 차차 알게 될 것이다).



위 그림에서 확인할 수 있듯이 중심 주파수 크기와 채널 번호 순서는 서로 동일하지 않으며, 일부 BLE 시스템의 경우는 2.402~2.48 GHz 대역 대신에 2.4~2.4835 GHz 대역을 사용하기도 한다.

## 2.2 Link layer (LL)Permalink

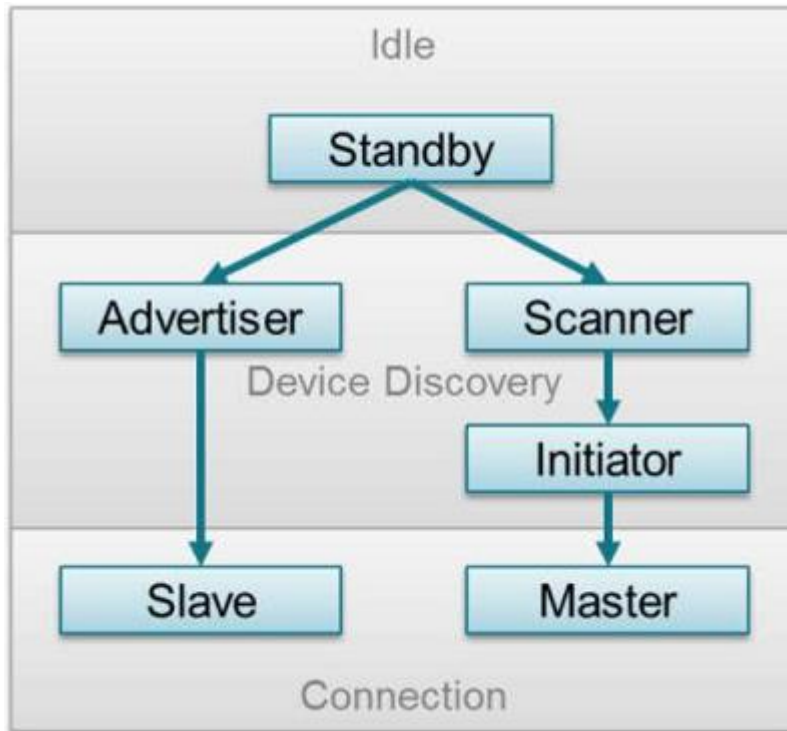
**LL** 는 **PHY** 와 직접적으로 상호작용하는 레이어이고, 하드웨어와 소프트웨어 사이에서 동작한다. 하드웨어와 연결되어 BLE 디바이스 간의 연결을 직접적으로 관리하는 레이어로 암호화 (**encryption**), 연결 상태 및 채널 업데이트 (**connection or channel update**) 등의 역할을 수행한다.



위 그림에서와 같이 BLE 디바이스는 LL 에서 다음 중 하나의 상태로 동작한다.

- Standby
- Advertising
- Scanning
- Initiating
- Connected

두 개의 BLE 디바이스가 연결을 시도할 때, 각각의 디바이스는 LL 에서 다음의 그림과 같이 동작한다.



출처:

[https://dev.ti.com/tirex/content/simplelink\\_cc2640r2\\_sdk\\_1\\_40\\_00\\_45/docs/blestack/ble\\_user\\_guide/html/ble-stack-3.x/gap.html](https://dev.ti.com/tirex/content/simplelink_cc2640r2_sdk_1_40_00_45/docs/blestack/ble_user_guide/html/ble-stack-3.x/gap.html)

먼저 BLE 연결을 시작하고자 하는 **master** 디바이스 (오른쪽)에서는 **Scanning** 동작을 수행한다. **Scanning**이란, 단어 뜻 그대로 주변에 BLE 연결이 가능한 디바이스가 있는지 확인하는 과정이다. 다음으로 BLE 연결을 대기하는 **slave** 디바이스의 경우 (왼쪽) **Advertising**이란 동작을 통해 해당 디바이스가 근처에 있다는 것을 주변에 알린다. 이후, **master** 디바이스에서 **Scanning**한 디바이스 중 하나에 연결을 시도할 경우, 해당 디바이스는 **LL**에서 연결을 준비하는 **initiating** 상태에 진입하게 되고, 무선 연결에 성공하고나면 **master**와 **slave** 디바이스는 모두 **Connected** 상태에 놓이게 된다.

이러한 BLE 연결 과정은 이후 설명할 **GAP**에 의해 관리된다.

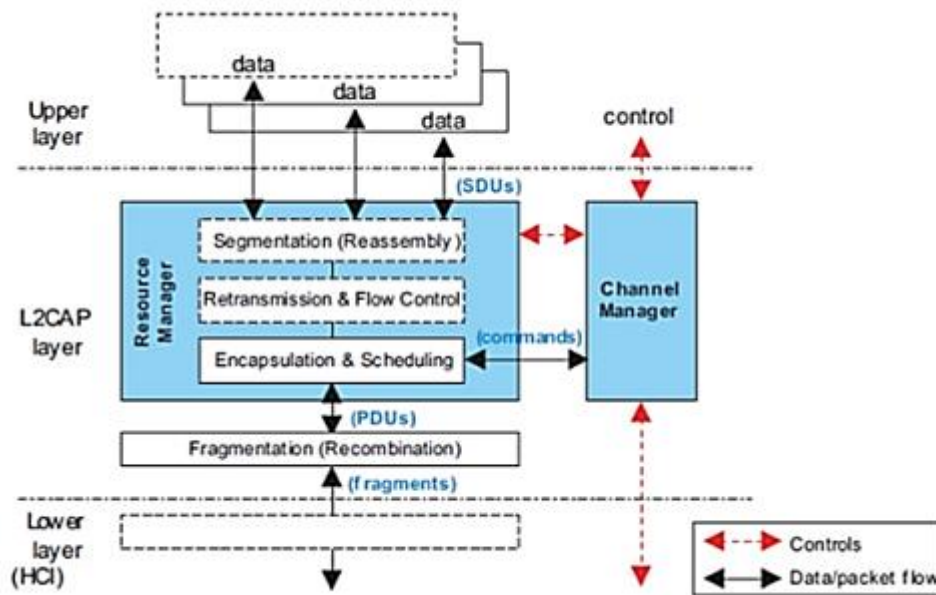
## 2.3 HCI, L2CAP, SMP, Permalink

### 2.3.1 HCI Permalink

**HCI** (Host-Controller Interface)는 **Host**와 **Controller**가 분리되어 있는 경우 두 모듈 사이의 상호작용을 담당하는 레이어이다. 앞으로 소개할 **Nordic Semiconductor**사의 **BLE** 프로세서를 비롯해, 대부분의 **BLE** 디바이스는 **Host**와 **Controller**가 단일 칩으로 구성되어 있다.

### 2.3.2 L2CAP Permalink

**L2CAP**는 **Logical Link Control and Adaptation Protocol**의 약자로, 상위 레이어 (**GATT/GAP**)와 하위 레이어 사이의 상호작용을 담당한다. **LL**에서 만들어진 **BLE** 패킷 (**Packet**)에 실제 어플리케이션 단에서 사용되는 데이터를 포함시키거나 추출하는 역할 등을 수행한다.



출처:

[https://dev.ti.com/tirex/content/simplelink\\_cc26x2\\_sdk\\_1\\_60\\_00\\_43/docs/ble5stack/ble\\_user\\_guide/html/ble-stack-5.x/l2cap.html](https://dev.ti.com/tirex/content/simplelink_cc26x2_sdk_1_60_00_43/docs/ble5stack/ble_user_guide/html/ble-stack-5.x/l2cap.html)

### 2.3.3 SMP

Security Manager 는 자주 연결하는 디바이스 (**peer**) 사이의 보안 코드 (**key**)를 관리하고 배포하는 역할을 수행한다.

## 2.4 Attribute Protocol (ATT)

**ATT** 는 서버 (**server**)와 클라이언트 (**client**) 사이의 데이터 교환에 대한 규칙으로, 무선으로 연결된 BLE 디바이스 사이의 데이터 교환은 **ATT** 에 의해 이뤄진다. 이때, 연결된 디바이스 중 센서의 값이나 스위치의 상태, 위치에 대한 정보 등을 가지고 있는 디바이스를 **GATT server** 라고 부르고, 해당 정보를 요청하는 디바이스를 **GATT client** 라고 부른다.

또한, **GATT** 는 Generic Attribute Profile 의 약자로 **ATT** 를 기반으로 주고받는 데이터의 구조를 정의한다. **ATT** 와 **GATT** 에 대해서는 다음 포스트에서 보다 구체적으로 다룰 예정이다.

## 2.5 Generic Access Profile (GAP)

**GAP** 는 연결 및 advertising 동작을 관리하는 최상위 레이어로 각 디바이스의 **LL** 상태를 결정하고, 서로 다른 BLE 디바이스 사이의 상호작용을 관리하는 모듈이다. BLE 디바이스는 **GAP** 에 대해 다음 중 하나의 역할 (**role**) 로 동작한다.

- Broadcaster
- Observer
- Peripheral

- **Central**

먼저, **Peripheral** (주변기기)로 동작하는 디바이스는 **LL** 에서 **Advertising** 상태를 유지하고 있으며, 다른 디바이스로부터의 연결 요청을 기다린다. 앞서 언급했듯이 **Advertising** 이란, 주변의 **BLE** 디바이스에 자신의 정보를 알리는 동작이고, 일반적으로 이 과정에서는 해당 디바이스의 이름과 간단한 정보만을 송신한다. 예를 들어, 아래의 그림과 같이 스마트폰에서 무선 이어폰과 연결하기 위해 근처 블루투스 기기를 찾고(**Scanning**) 있다고 가정해보자. 이때 본인의 이어폰 (e.g. AirPods)을 포함한 다수의 블루투스 기기의 이름이 보일텐데, 이 때 감지되는 디바이스들은 모두 **Advertising** 상태에 있는 **Peripheral** 디바이스라고 할 수 있다.



출처: <https://www.semiconductorstore.com/blog/2017/Bluetooth-5-versus-Bluetooth-4-2-whats-the-difference/2080>

위 예시에서의 스마트폰과 같이 **Peripheral** 디바이스를 **Scanning** 하고, 그 중 하나의 디바이스와 연결까지 가능한 디바이스는 **GAP** 에서 **Central** (중앙장치)로서 동작하는 디바이스이다.

**BLE 연결은 Central 과 Peripheral 디바이스 사이에서 이뤄지고, Central 디바이스에 의해 시작된다.**

반면에 **Peripheral** 디바이스가 연결 과정 없이 **Advertising** 동작만을 수행하는 경우, 이러한 디바이스를 **Broadcaster** 라고 한다. 기회가 되면 설명하겠지만, **Broadcaster** 로 동작하는 디바이스는 **Central** 디바이스에서도 연결할 수 없으며, **Advertising** 데이터에 자신 이름 이외에도 실질적인 데이터를 실어서 전송한다.

**일반적으로 이러한 장치를 비콘(Beacon)이라고 부른다.**

또한, **Central** 디바이스가 연결 과정 없이 **Scanning** 동작만을 수행하는 경우 해당 디바이스를 **Observer** 라고 부르며, **Observer** 의 경우 일반적으로 **Advertising** 패킷 내의 데이터를 타겟으로 하므로 **Beacon** 신호를 수신하고자 할 때 사용한다.

---

## Reference

[https://dev.ti.com/tirex/content/simplelink\\_cc2640r2\\_sdk\\_1\\_40\\_00\\_45/docs/blestack/ble\\_user\\_guide/html/ble-stack-3.x/gap.html](https://dev.ti.com/tirex/content/simplelink_cc2640r2_sdk_1_40_00_45/docs/blestack/ble_user_guide/html/ble-stack-3.x/gap.html)

[https://dev.ti.com/tirex/content/simplelink\\_cc26x2\\_sdk\\_1\\_60\\_00\\_43/docs/ble5stack/ble\\_user\\_guide/html/ble-stack-5.x/l2cap.html](https://dev.ti.com/tirex/content/simplelink_cc26x2_sdk_1_60_00_43/docs/ble5stack/ble_user_guide/html/ble-stack-5.x/l2cap.html)

<https://www.semiconductorstore.com/blog/2017/Bluetooth-5-versus-Bluetooth-4-2-whats-the-difference/2080>